

NT409 — QUANTUM COMPUTING AND APPLICATIONS

Suggested Course Project Topics

Faculty of Computer Networks and Communications — Department of Computer Networks

VNUHCM — University of Information Technology

Instructors: Phan Trung Phat, M.Sc. (phatpt@uit.edu.vn)

1. How to Use This Document

This catalogue proposes twenty project topics for the NT409 course project. Every topic is scoped so that a team of two or three students can complete it in roughly ten weeks alongside their other coursework. Teams may also propose their own topic, provided it is agreed with the instructors and meets the same scope and evaluation requirements.

Choosing well: a narrow question answered thoroughly always scores better than an ambitious question answered superficially.

2. Working Within Hardware Access Limits

There is no quantum computer in Vietnam, and access to overseas machines is limited, queued and often metered. Every project in this catalogue is therefore designed to be completed on a classical simulator. Real hardware, where it is used at all, is treated as an optional final validation step rather than a dependency.

The practical implications for your planning:

- A laptop simulator handles roughly 20-25 qubits comfortably, and up to about 30 with sufficient RAM. Nearly every project here needs fewer than 15, so simulation is not the limiting factor.
- Noisy simulation using a noise model imported from a real device reproduces hardware behaviour closely enough for coursework, without any queue or cost.
- Free-tier cloud access does exist. IBM Quantum Platform provides limited free QPU time, and Amazon Braket and Azure Quantum offer free simulator tiers plus starting credits. Register early, because queue times from Vietnam can run to hours.
- Google Colab is a reliable fallback if a team member lacks a capable machine, and it handles every project in this catalogue.
- Never let your final result depend on a hardware run completing. Produce your complete results on a simulator first, then add hardware data as a bonus if the queue delivers in time.

Calibration data: real device physics without the queue

Cloud providers publish, and continuously update, the measured physical properties of every device they operate: relaxation and dephasing times (T_1 and T_2), per-gate error rates, gate durations, readout assignment errors and the qubit connectivity graph. This data is retrievable through the provider API without submitting a job, so it costs no queue time, no credits and no quota — yet it carries the real measured physics of a specific machine on a specific day.

- Assembled together, those parameters form a digital twin: a software replica of a named backend that accepts ordinary circuits and predicts both output and runtime. Group A builds and validates these twins; everything downstream uses them.

3. Project Index

#	Project Title	Category	Level
1	Circuit Depth and Transpilation Across Backends	Circuits	Easy
2	Calibration Data Explorer and Device Health Dashboard	Digital Twin & Calibration Data	Easy
3	Noise-Aware Qubit Selection and Layout Optimisation	Digital Twin & Calibration Data	Medium
4	Cross-Backend Performance Prediction from Calibration Data	Digital Twin & Calibration Data	Advanced
5	Quantum Job Scheduling Simulation with QSimPy	Resource Management	Medium
6	Task Placement Policies: Heuristics versus Deep Reinforcement Learning	Resource Management	Advanced
7	Quantum Cloud Access Workflow and Performance Benchmarking	Systems & Architecture	Easy
8	Multi-Tenant Qubit Allocation and Circuit Multi-Programming	Systems & Architecture	Medium - Advanced
9	Grover Search on a Small Database	Algorithms	Medium
10	QAOA for Combinatorial Optimisation with Classical Baselines	Optimisation	Medium
11	Variational Quantum Eigensolver (VQE) on a Simple Cost Function	Optimisation	Medium
12	QUBO Formulation and Penalty Design for Resource Allocation	Optimisation	Medium
13	Quantum Network Simulation	Networking	Advanced
14	Quantum Approach to Small Routing Problems	Networking	Medium
15	Transmission Scheduling in a Small Network	Networking	Medium
16	Task Placement in a Small Cluster	Cloud	Medium
17	Load Balancing as a Number-Partitioning QUBO	Cloud	Easy
18	Post-Quantum Cryptography Benchmarking and Migration Assessment	Security & Quantum Cryptography	Easy
19	BB84 Quantum Key Distribution Simulation	Security & Quantum Cryptography	Easy
20	Shor's Algorithm and Cryptographic Resource Extrapolation	Security & Quantum Cryptography	Advanced
21	Quantum Classifier on a Small Dataset	Quantum Machine Learning	Medium

#	Project Title	Category	Level
22	Quantum Feature Maps and Data Encoding Strategies	Quantum Machine Learning	Medium

4. Project Descriptions

Project 1: Circuit Depth and Transpilation Across Backends

Category: Circuits

Difficulty Level: Easy

Short Description: A circuit written abstractly must be rewritten into a device native gate set and mapped onto its qubit connectivity. This project measures how much that rewriting inflates circuit size, which in practice is the single biggest constraint on what a NISQ device can run.

Implementation Approach:

1. Select four or five benchmark circuits: a QFT, a GHZ state, a small Grover instance, and one or two random circuits of fixed depth.
2. Transpile each against several coupling maps — linear, IBM heavy-hex, and fully connected — at optimisation levels 0 through 3.
3. Record depth, total gate count, two-qubit (CX) gate count, and the number of SWAP gates the router had to insert.
4. Present results as tables and grouped bar charts, showing which circuits suffer most from limited connectivity.
5. Combine your depth numbers with the per-gate error rates measured in project A2 to estimate the realistic depth budget on today devices.

Tools & Software: Qiskit transpiler, fake backends from qiskit-ibm-runtime, pandas for result tables, Matplotlib.

Project 2: Calibration Data Explorer and Device Health Dashboard

Category: Digital Twin & Calibration Data

Difficulty Level: Easy

Short Description: Quantum cloud providers publish detailed calibration records for every device they operate, yet this data is rarely examined systematically by students. This project builds an exploratory analysis pipeline and interactive dashboard that retrieves, cleans and visualises device properties across multiple backends, producing a shared class reference for what present-day hardware actually looks like.

Implementation Approach: The team begins by authenticating against a provider API and retrieving backend property records covering relaxation and dephasing times, single- and two-qubit gate error rates, gate durations, readout assignment errors and the qubit coupling map, storing each retrieval as a timestamped snapshot so that the work remains reproducible. These raw records are normalised into a tidy tabular schema in which every row describes one qubit or one gate on one device at one moment in time, which makes cross-device comparison straightforward and exposes missing or anomalous entries that must be handled explicitly rather than silently dropped. Analysis then proceeds through descriptive

statistics and distribution plots characterising the spread of qubit quality within a single device and across the provider fleet, supported by connectivity graph visualisations in which node and edge colouring encode measured error rates. The final deliverable wraps these views in a lightweight interactive dashboard allowing a user to select a backend and immediately identify its weakest qubits, its best-connected regions and its standing relative to peer devices.

Tools & Software: Python, Qiskit, qiskit-ibm-runtime, pandas, NumPy, Matplotlib, seaborn, NetworkX, Streamlit or Plotly Dash, Jupyter

Project 3: Noise-Aware Qubit Selection and Layout Optimisation

Category: Digital Twin & Calibration Data

Difficulty Level: Medium

Short Description: Physical qubits on a real device differ substantially in quality, so the choice of which qubits a circuit occupies can influence the outcome more than the algorithm itself. This project builds a calibration-driven layout selector and measures how much it improves upon default compiler heuristics.

Implementation Approach: The team constructs a weighted device graph in which edges carry measured two-qubit gate error and nodes carry readout and coherence quality, forming the substrate for all subsequent scoring. A cost function is then designed that estimates the survival probability of a given circuit under a candidate physical layout by combining the circuit's gate counts with the corresponding per-gate error rates, deliberately kept simple enough to evaluate over many candidate placements quickly. Layout search is handled exhaustively for small circuits and by an error-weighted subgraph matching heuristic for larger ones, and the resulting selections are compared against the compiler's own layout passes across a benchmark suite, with noisy simulation used to confirm that predicted improvements are actually realised in measured output. The study closes by re-scoring the chosen layouts against calibration data recorded on a different date, testing whether a layout selected today remains a sound choice a week later and thereby connecting the result to the question of calibration drift.

Tools & Software: Python, Qiskit transpiler and layout passes, qiskit-ibm-runtime, rustworkx or NetworkX, Qiskit Aer, pandas, Matplotlib.

Project 4: Cross-Backend Performance Prediction from Calibration Data

Category: Digital Twin & Calibration Data

Difficulty Level: Advanced

Short Description: Given a circuit and a choice of several devices, which one should you run it on? This project builds and validates a predictive model that answers that question from calibration data alone, without spending queue time to find out empirically.

Implementation Approach:

1. Assemble calibration data for as many backends as you can reach, plus a benchmark circuit suite spanning several structures and depths.
2. Define candidate predictors: a naive product of gate errors, an error-per-layered-gate style metric, and a regression model fitted over calibration features.

3. Generate ground truth by noisy simulation using each backend calibration-derived model, and validate a subset against real hardware runs where free-tier access permits.
4. Evaluate predictors by rank correlation rather than absolute error, since the practical question is which device is best, not what the exact fidelity will be.
5. Deliver a backend recommendation tool together with an honest error analysis stating where the prediction fails and why.

Tools & Software: qiskit-ibm-runtime, qiskit-aer, scikit-learn, pandas, SciPy, Matplotlib.

Project 5: Quantum Job Scheduling Simulation with QSimPy

Category: Resource Management

Difficulty Level: Medium

Short Description: QSimPy is a discrete-event simulation framework purpose-built for quantum cloud resource management, in which quantum nodes are modelled from real system calibration data and workloads from actual circuit features. This project establishes a working simulation testbed and characterises how scheduling outcomes depend upon the device parameters supplied to it.

Implementation Approach: The team first installs the framework and reproduces its bundled example so that a known-good baseline exists before any modification is attempted, which matters more than usual here because the toolkit is research software rather than a production library. Default node parameters are then replaced with values retrieved from live backend calibration, and the mapping from each calibration field to its corresponding node attribute is documented carefully, since this correspondence is where most modelling errors originate and where a reader will most want to check the work. Workload construction follows, beginning from the provided benchmark task datasets and extending them with circuits the team transpiles themselves in order to obtain realistic width, depth and gate-count features. Simulation experiments then compare a small set of scheduling policies under identical workloads, and the study concludes with a sensitivity analysis in which nodes are rebuilt from calibration snapshots of differing quality, establishing which device attributes genuinely drive simulated outcomes and which are largely inconsequential.

Quantum cloud resource management toolkits, all of which model quantum systems as schedulable resources and quantum circuits as workload entities:

Toolkit	Base	Focus
QSimPy	Python / SimPy + Gymnasium	Learning-centric quantum cloud resource management; QNodes from calibration data (main catalogue Group B)
iQuantum	Java / CloudSim	Task scheduling, backend selection, hybrid orchestration and offloading across the quantum cloud-edge continuum

Tools & Software: Python, QSimPy, SimPy, qiskit-ibm-runtime, Qiskit, pandas, Matplotlib.

Project 6: Task Placement Policies: Heuristics versus Deep Reinforcement Learning

Category: Resource Management

Difficulty Level: Advanced

Short Description: Deciding which quantum processor should execute each incoming job is the central resource management problem in quantum cloud computing, and simulation frameworks now expose it as a reinforcement learning environment. This project implements both hand-designed and learned placement policies and evaluates them under strictly equivalent conditions.

Implementation Approach: The problem is first specified precisely as a sequential decision process, with an explicit state representation summarising node availability and quality, an action space over candidate nodes, and a reward function combining completion time, achieved fidelity and any rescheduling penalty; this specification is fixed before any agent is trained so that heuristic and learned policies demonstrably address an identical problem. Strong heuristic baselines are implemented next, including greedy fastest-node, least-queue and best-fidelity rules, because a learned policy unable to outperform a well-tuned heuristic has established nothing of value. Agents are then trained using established deep reinforcement learning implementations, with learning curves logged across multiple random seeds rather than a single fortunate run, and evaluation conducted on held-out workloads with results reported as means and variances. The investigation closes by interrogating what the trained agent actually learned, examining which node features drive its decisions and whether its advantage survives a deliberate shift in the workload distribution.

Tools & Software: Python, QSimPy, Gymnasium, Ray RLlib, PyTorch, TensorBoard, pandas, Matplotlib.

Project 7: Quantum Cloud Access Workflow and Performance Benchmarking

Category: Systems & Architecture

Difficulty Level: Easy

Short Description: Executing a circuit on real hardware involves authentication, transpilation, queueing, execution and result retrieval, each introducing practical constraints invisible from a local simulator. This project documents the complete workflow end to end and produces an access guide for future cohorts working under local connectivity and quota limitations.

Implementation Approach: The work begins by registering for free-tier accounts across at least two providers and recording the practical details of that process, including quota limits, authentication mechanisms and any regional restrictions encountered, since this information is genuinely useful to subsequent students and is nowhere documented for this context. A small benchmark circuit comfortably within free-tier limits is then executed through three distinct paths, namely a local simulator, a cloud-hosted simulator and a real quantum processor, with matched shot counts and repeated submissions at different times of day so that queue behaviour is characterised statistically rather than anecdotally. Instrumentation captures wall-clock time, queue delay, transpilation overhead and any credit consumption, while the returned distributions are compared quantitatively to establish the discrepancy introduced by real hardware relative to idealised simulation. The deliverable is a reproducible notebook together with a written access guide covering registration, quota management and realistic expectations for turnaround time.

Tools & Software: Python, Qiskit, qiskit-ibm-runtime, Amazon Braket SDK, Qiskit Aer, pandas, Matplotlib, Jupyter.

Tools & Software: Python, PyTorch, TensorBoard, pandas, Matplotlib.

Project 8: Multi-Tenant Qubit Allocation and Circuit Multi-Programming

Category: Systems & Architecture

Difficulty Level: Medium - Advanced

Short Description: Contemporary quantum processors provide considerably more qubits than a small circuit requires, so several independent circuits can in principle share one device concurrently. This project investigates how disjoint qubit regions should be allocated among competing tenants and what fidelity penalty co-location imposes.

Implementation Approach: Allocation is formulated over a calibration-weighted device graph, where the objective is to assign each concurrent circuit a connected region of sufficient size and quality while respecting mutual disjointness. Several allocation strategies are implemented, ranging from a quality-greedy approach that assigns the best available region to each arriving circuit through to a partitioning-based scheme that reserves high-quality regions equitably, with random assignment retained throughout as an experimental control. Each circuit's fidelity is then measured both in isolation and under co-location using a calibration-derived noise model, allowing the degradation attributable specifically to sharing to be isolated cleanly from the device's baseline error rate. The analysis examines the trade-off between aggregate throughput and per-circuit fidelity as packing density increases, identifying a practical operating point, and closes by evaluating whether any allocator systematically disadvantages particular tenants, together with a fairness-aware variant designed to address that concern.

Tools & Software: Python, Qiskit, Qiskit Aer, qiskit-ibm-runtime, rustworkx or NetworkX, NumPy, pandas, Matplotlib

Project 9: Grover Search on a Small Database

Category: Algorithms

Difficulty Level: Medium

Short Description: Builds Grover amplitude amplification for a small unstructured search problem and studies how the success rate depends on the number of iterations, including the counter-intuitive result that more iterations can make things worse.

Implementation Approach:

1. Encode a search problem over three to five qubits, for example finding the satisfying assignment of a small constraint or SAT instance.
2. Build the phase oracle that marks solutions and the diffusion operator that amplifies them.
3. Sweep the iteration count from zero past the theoretical optimum and record the measured success probability at each point.
4. Confirm the characteristic oscillation and compare your measured optimum against the predicted value.
5. Repeat under a noise model and report how the success peak flattens and shifts, then state the largest instance that remains reliable.

Tools & Software: Qiskit (including the Grover operator utilities), qiskit-aer, NumPy, Matplotlib.

Project 10: QAOA for Combinatorial Optimisation with Classical Baselines

Category: Optimisation

Difficulty Level: Medium

Short Description: Solves graph partitioning with the Quantum Approximate Optimisation Algorithm, the flagship hybrid NISQ algorithm, and benchmarks the solution quality against established classical heuristics.

Implementation Approach:

1. Generate small graphs of six to twelve nodes with networkx and compute the exact maximum cut by brute force so you have ground truth.
2. Build the QAOA cost and mixer Hamiltonians and the parameterised circuit at depth $p = 1, 2$ and 3 .
3. Optimise the circuit parameters with a classical optimiser such as COBYLA or SPSA, logging the full optimisation trace.
4. Record the approximation ratio as a function of depth p and of graph size.
5. Compare against a greedy heuristic, simulated annealing, and uniform random sampling as a floor; state clearly whether QAOA beat any of them at this scale.

Tools & Software: Qiskit with qiskit-optimization, or PennyLane; networkx, SciPy optimisers, Matplotlib.

Project 11: Variational Quantum Eigensolver (VQE) on a Simple Cost Function

Category: Optimisation

Difficulty Level: Medium

Short Description: Sets up the Variational Quantum Eigensolver loop and studies its convergence behaviour: how the choice of ansatz and optimiser matters, and how shot noise interferes with the classical optimiser.

Implementation Approach:

1. Define a small Hamiltonian — a two to four qubit Ising or Heisenberg chain, or the H₂ molecule at a fixed bond length if you want a chemistry flavour.
2. Compute the exact ground-state energy by direct diagonalisation to serve as your reference value.
3. Implement two or three ansatz circuits of differing depth and expressibility.
4. Run VQE with several optimisers (COBYLA, SPSA, Adam) and several shot budgets, recording energy against iteration for each combination.
5. Plot the convergence curves, report the final error against the exact value, and discuss any barren-plateau symptoms such as vanishing gradients on the deeper ansatz.

Tools & Software: PennyLane (automatic differentiation) or Qiskit, SciPy, NumPy, Matplotlib.

Project 12: QUBO Formulation and Penalty Design for Resource Allocation

Category: Optimisation

Difficulty Level: Medium

Short Description: Translating a constrained resource allocation problem into the unconstrained binary form required by quantum optimisers is where most practical difficulty resides, and penalty weight selection determines whether returned solutions are feasible at all. This project studies that translation systematically.

Implementation Approach:

1. Pick a concrete assignment problem at 3x3 to 5x5 size — tasks to workers, students to projects, or lectures to rooms.
2. Write the objective and the hard constraints, then fold the constraints into the objective as penalty terms.
3. Derive and tune the penalty weight so that no infeasible solution can ever score better than a feasible one; document how you chose it.
4. Convert the QUBO to an Ising Hamiltonian and solve it on a simulator with QAOA or VQE.
5. Measure the feasibility rate as a function of penalty weight — this is your key experimental result — and validate optimality against brute force or a classical MILP solver.

Tools & Software: Qiskit with qiskit-optimization, NumPy, PuLP or Google OR-Tools for the classical baseline, docplex (optional).

Project 13: Quantum Network Simulation

Category: Networking

Difficulty Level: Advanced

Short Description: Delivering an end-to-end entangled pair requires generating link-level entanglement and swapping it along a path, so path selection matters as much as in classical routing — but fidelity decays at every hop, which makes hop count the wrong metric. This project implements and benchmarks routing policies on a simulated quantum network.

Implementation Approach: Test and evaluate one of these simulation tools:

Simulator	Base	Best suited to
SeQUeNCe	Python, discrete-event	Full-stack simulation of quantum links, systems and networks; modular design
NetSquid	Python, discrete-event	Physically realistic device models with accurate timing; effects of time on device performance
QuISP	C++ on OMNeT++	Large repeater networks; tracks an error model rather than full quantum state, so it scales
SimQN	Python + Cython	Network-layer investigation, architecture-agnostic; reported faster than NetSquid
QuNetSim	Python	Small networks (roughly 5-10 nodes), OSI-inspired layering, upper-layer application protocols

Tools & Software: Python, PyTorch, TensorBoard, pandas, Matplotlib.

Project 14: Quantum Approach to Small Routing Problems

Category: Networking

Difficulty Level: Medium

Short Description: Models a routing or shortest-path decision on a small network as a binary optimisation problem and evaluates honestly whether a quantum formulation is workable at a scale a simulator can handle.

Implementation Approach:

1. Build a small topology of six to ten nodes with link weights representing latency or cost.
2. Formulate path selection as a QUBO, encoding flow-conservation constraints as penalty terms.
3. Solve with QAOA on a simulator, then decode the best measured bitstring back into an actual path.
4. Validate against Dijkstra on the same graph and report how often the quantum result matches the true shortest path.
5. Report the qubit requirement as a function of node and edge count, then extrapolate what topology size current hardware could realistically handle.

Tools & Software: Qiskit, networkx (Dijkstra baseline), NumPy, Matplotlib.

Project 15: Transmission Scheduling in a Small Network

Category: Networking

Difficulty Level: Medium

Short Description: Formulates a link or transmission scheduling problem, where conflicting transmissions must not share a slot, and solves it as a graph colouring or maximum independent set problem using QAOA.

Implementation Approach:

1. Model the scheduling conflict as a graph in which nodes are transmissions and edges mark pairs that cannot be scheduled together.
2. Reduce the problem to Maximum Independent Set or graph colouring, then express it as a QUBO.
3. Solve with QAOA at several circuit depths on a simulator.
4. Compare the resulting schedule length or throughput against a greedy scheduler, and against the exact optimum for the smallest instances.
5. Analyse how the qubit requirement grows with the number of links multiplied by time slots, and state the practical ceiling.

Tools & Software: Qiskit, networkx, NumPy, Matplotlib.

Project 16: Task Placement in a Small Cluster

Category: Cloud

Difficulty Level: Medium

Short Description: Models virtual machine or container placement onto hosts under capacity constraints as a quantum optimisation problem, and compares the outcome against the bin-packing heuristics that real schedulers actually use.

Implementation Approach:

1. Define a small placement instance: four to eight tasks with CPU and memory demands, and two to four hosts with fixed capacities.
2. Formulate it as a QUBO with one-hot assignment constraints and capacity penalty terms.
3. Solve with QAOA on a simulator and decode the resulting placement.
4. Benchmark against First-Fit, Best-Fit Decreasing and an exact MILP solution on the same instances.
5. Report solution quality, feasibility rate and qubit count, and state at what instance size classical simulation stops being tractable on your laptop.

Tools & Software: Qiskit, NumPy, Google OR-Tools or PuLP for baselines, Matplotlib.

Project 17: Load Balancing as a Number-Partitioning QUBO

Category: Cloud

Difficulty Level: Easy

Short Description: Expresses load distribution across servers as a number-partitioning QUBO and studies systematically how solution quality degrades as the problem grows. A well-bounded project whose negative results are still publishable within the course.

Implementation Approach:

1. Generate load sets of increasing size (6, 8, 10 and 12 jobs) whose optimal split you can compute exactly.
2. Write the number-partitioning QUBO that minimises the imbalance between two or more servers.
3. Solve each instance size with QAOA at a fixed depth and record the imbalance achieved.
4. Plot solution quality against problem size and against QAOA depth, alongside the greedy Longest-Processing-Time baseline.
5. Identify the size at which simulation time rather than solution quality becomes the binding constraint, and report it as a headline finding.

Tools & Software: Qiskit, NumPy, pandas, Matplotlib.

Project 18: Post-Quantum Cryptography Benchmarking and Migration Assessment

Category: Security & Quantum Cryptography

Difficulty Level: Easy

Short Description: Standardised post-quantum algorithms are now available in reference implementations, and their performance characteristics differ substantially from the classical schemes they are intended to replace. This project measures those differences and assesses the practical cost of migration.

Implementation Approach: Reference implementations of several standardised schemes spanning different mathematical families are installed and exercised, with correct key generation, encapsulation, signing and verification confirmed before any measurement is attempted, since benchmarking a misconfigured implementation produces confident and entirely meaningless numbers. Systematic benchmarking then records public key sizes, ciphertext and signature sizes, and operation timings, with every measurement repeated many times so that distributions rather than single timings are reported, cryptographic operation timings being noticeably variable in practice. Direct comparison against established classical schemes executed on identical hardware places these results in context and renders the size and speed trade-offs concrete rather than abstract. The study closes by modelling the impact upon a specific deployment scenario, such as the growth in handshake size for a secure connection or the feasibility of deployment upon a constrained embedded device, thereby translating raw benchmarks into an operational assessment.

Tools & Software: Python, liboqs and liboqs-python, OpenSSL, pandas, NumPy, Matplotlib.

Project 19: BB84 Quantum Key Distribution Simulation

Category: Security & Quantum Cryptography

Difficulty Level: Easy

Short Description: BB84 demonstrates that quantum mechanics permits two parties to detect eavesdropping upon their key exchange, a property with no classical analogue. This project implements the protocol end to end and verifies its security claims experimentally rather than merely describing them.

Implementation Approach: The protocol is implemented in full, covering random bit and basis generation, state preparation, transmission, measurement in randomly chosen bases, public basis reconciliation and key sifting, with correctness established first in an idealised setting where the sifted keys held by both parties must agree exactly. An intercept-resend eavesdropper is then introduced and the resulting quantum bit error rate measured across many trials, confirming convergence toward the theoretically predicted value and demonstrating that an eavesdropper's presence is detectable from error statistics alone without any additional assumption. Channel noise is added subsequently, raising the practically important question of how a legitimate but imperfect channel is to be distinguished from an attacked one, which the team addresses by characterising the error rate distribution under each condition and identifying a defensible detection threshold. Basic error correction and privacy amplification are implemented as a concluding extension to complete the protocol.

Tools & Software: Python, Qiskit or NumPy, SciPy, Matplotlib, Jupyter.

Project 20: Shor's Algorithm and Cryptographic Resource Extrapolation

Category: Security & Quantum Cryptography

Difficulty Level: Advanced

Short Description: Shor's algorithm threatens widely deployed public key cryptography, yet the gap between laboratory demonstration and practical attack is enormous. This project executes the algorithm at simulable scale and constructs a defensible extrapolation toward cryptographically relevant problem sizes.

Implementation Approach: The algorithm is implemented for the small composite numbers a simulator can accommodate, with correctness verified by recovering the expected factors, and the period-finding subroutine examined in detail so that the precise origin of the quantum speedup is understood rather than merely invoked as received wisdom. Resource consumption is then instrumented carefully, recording qubit count and circuit depth for each instance and fitting the observed scaling against the bit length of the number being factored, which yields an empirical growth law rather than a quoted asymptotic. This measured scaling is compared against published fault-tolerant resource estimates for cryptographically relevant key sizes, and the substantial resulting discrepancy is analysed in terms of the error correction overhead separating logical from physical qubits. The project concludes with an assessment of migration timelines and the implications for data requiring long-term confidentiality, connecting a technical result to operational security planning.

Tools & Software: Python, Qiskit, Qiskit Aer, SymPy, NumPy, Matplotlib.

Project 21: Quantum Classifier on a Small Dataset

Category: Quantum Machine Learning

Difficulty Level: Medium

Short Description: Variational quantum classifiers are frequently presented with encouraging results that do not survive comparison against well-tuned classical models. This project constructs such a classifier and evaluates it under methodologically rigorous conditions in which any claimed advantage must be defensible.

Implementation Approach: A small tabular dataset is selected and dimensionally reduced to a feature count compatible with available qubit budgets, with a test partition isolated at the outset and left entirely untouched throughout development in order to prevent the optimistic bias arising from repeated evaluation against the same data. Both a variational classifier and a quantum kernel method are implemented so that two distinct paradigms are represented rather than a single architecture whose weaknesses might be idiosyncratic, and training proceeds under cross-validation with hyperparameter search conducted within each fold. Classical baselines including regularised linear models, kernel support vector machines and modest neural networks are trained under identical partitions and comparable search budgets, since an under-tuned baseline invalidates any conclusion subsequently drawn from the comparison. Evaluation reports accuracy alongside its variance across folds, applies statistical testing to any observed difference, and examines learning curves as training set size varies.

Tools & Software: Python, PennyLane or Qiskit Machine Learning, scikit-learn, NumPy, SciPy, pandas, Matplotlib.

Project 22: Quantum Feature Maps and Data Encoding Strategies

Category: Quantum Machine Learning

Difficulty Level: Medium

Short Description: The mechanism by which classical data is embedded into quantum states frequently constrains quantum machine learning performance more severely than the trainable model does. This

project compares encoding strategies under a fixed qubit budget and characterises their practical consequences.

Implementation Approach: Several encoding schemes are implemented for a common dataset, spanning basis encoding, angle encoding, amplitude encoding and a higher-order feature map, with the qubit requirement, post-transpilation circuit depth and necessary classical preprocessing recorded for each so that their true costs become visible rather than remaining implicit. Quantum kernel matrices induced by each encoding are then computed and visualised, permitting direct inspection of whether an embedding separates the classes before any model is trained at all, which constitutes a considerably more informative diagnostic than downstream accuracy considered alone. Identical downstream classifiers are subsequently trained upon each encoding under matched conditions, isolating the contribution of the embedding itself from that of the trainable component. The study concludes by relating observed performance to expressibility and trainability considerations and recommending an encoding appropriate to a stated qubit budget and data dimensionality.

Tools & Software: Python, PennyLane or Qiskit, scikit-learn, NumPy, pandas, Matplotlib.